

Computer Architecture

Superscalar II

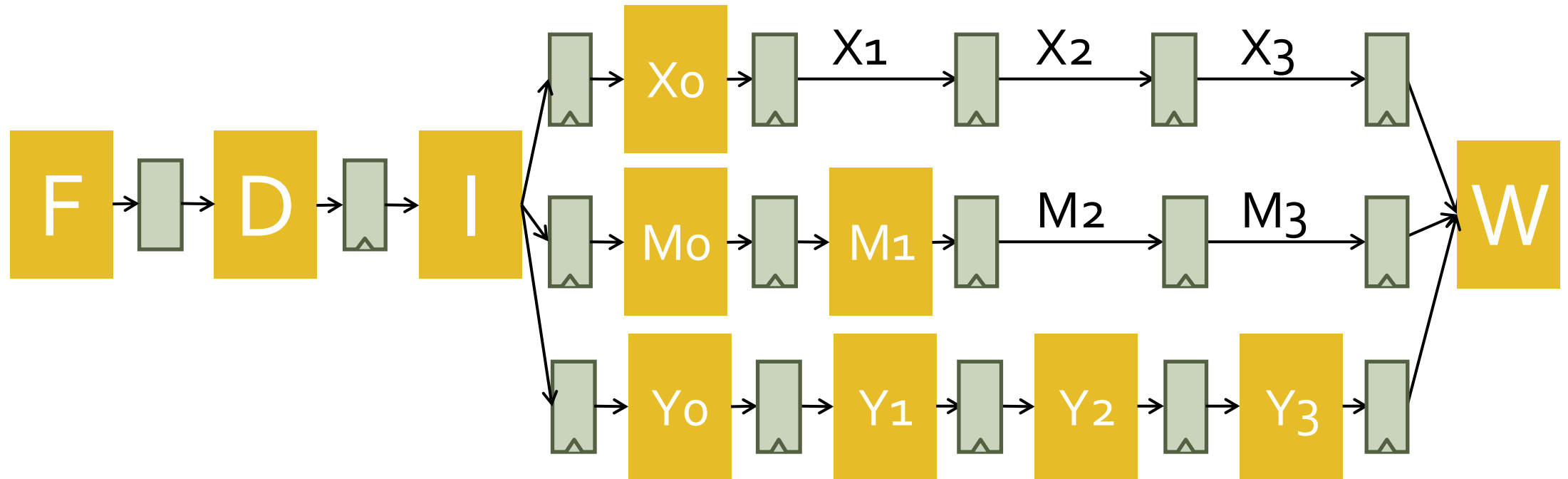
Ting-Jung Chang

NYCU CS

Recap: Out-Of-Order (OOO)

Name	Frontend	Issue	Writeback	Commit	
I ₄	IO	IO	IO	IO	Fixed Length Pipelines Scoreboard
I ₂ O ₂	IO	IO	OOO	OOO	Scoreboard
I ₂ O ₁	IO	IO	OOO	IO	Scoreboard, Reorder Buffer, and Store Buffer
IO ₃	IO	OOO	OOO	OOO	Scoreboard and Issue Queue
IO ₂ I	IO	OOO	OOO	IO	Scoreboard, Issue Queue, Reorder Buffer, and Store Buffer

Recap: I4



ARF

SB

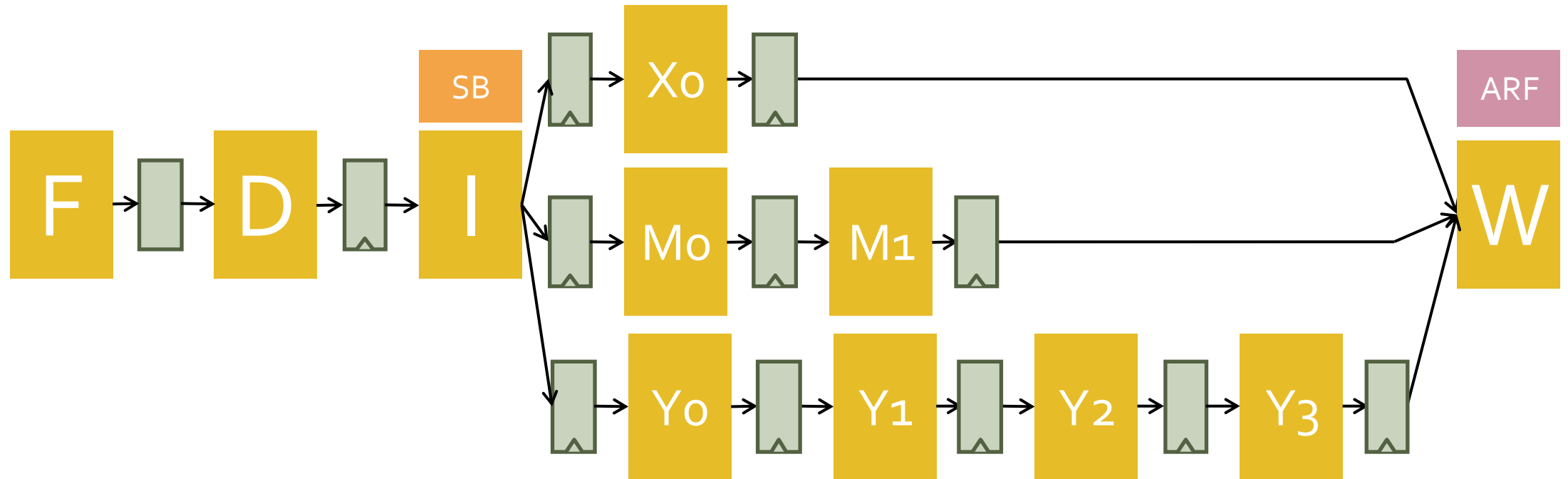
R

R/W

W

W

Recap: I2O2



Early Commit Point?

0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	/
1	addi	x11,x10,1		F	D	I	X0	W		/
2	mul	x5, x1, x4			F	D	I	I	I	/
3	mul	x7, x5, x6				F	D	D	D	/
4	addi	x12,x11,1					F	F	F	/
5	addi	x13,x12,1								/
6	addi	x14,x12,2								

- Limits certain types of exceptions

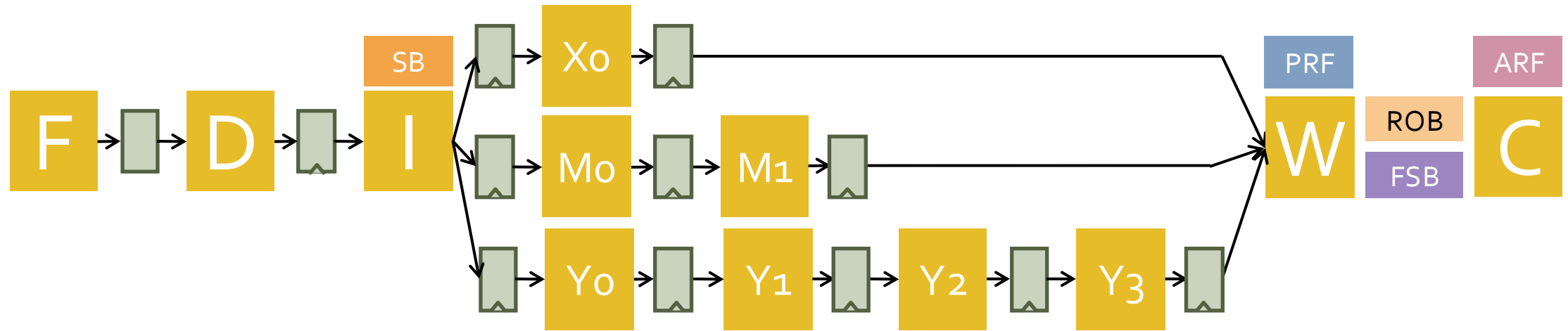
Course Admin

- PS1 solutions are out
- PS2 is out
- Lab2 is out

Agenda

- Out-of-Order Processors
- Speculation and Branches
- Register Renaming
- Memory Disambiguation

I2OI: In-order Frontend/Issue, Out-of-order Writeback, In-order Commit

[illegible]

PRF=Physical Register File(Future File), ROB=Reorder Buffer, FSB=Finished Store Buffer (1 entry)

Reorder Buffer (ROB)

State	S	ST	V	Preg
--				
P	1			
F	1			
P	1			
P				
F				
P				
--				
--				

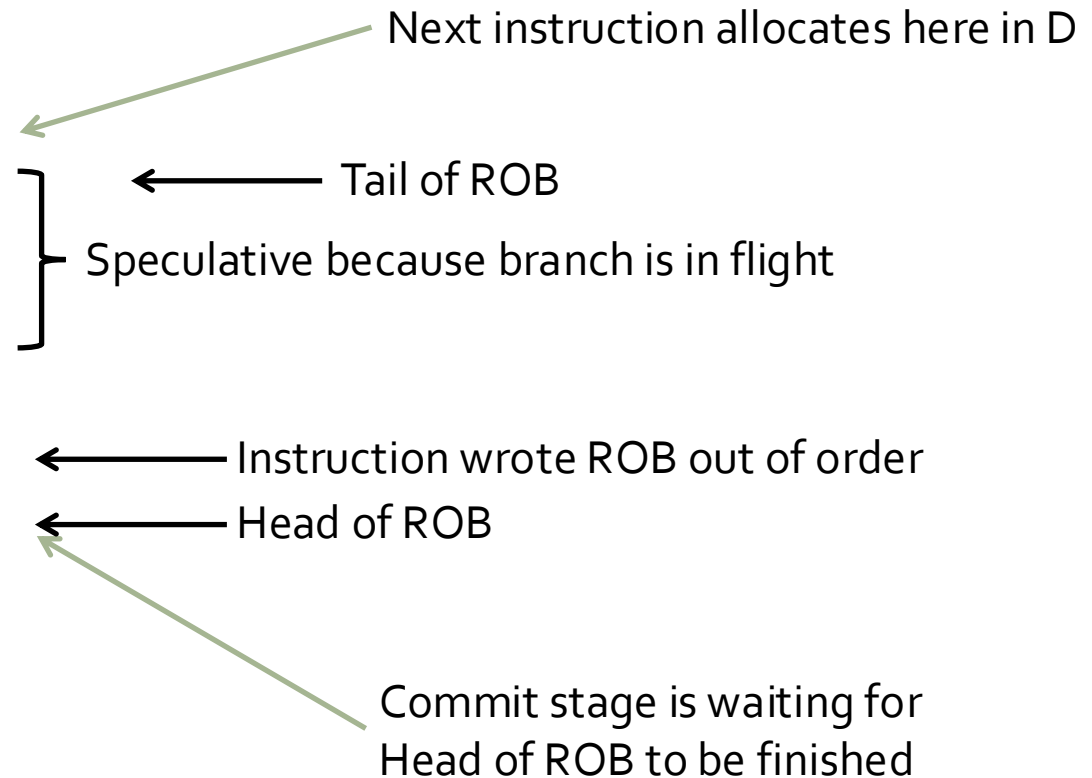
State: {Empty (--), Pending, Finished}

S: Speculative

ST: Store bit

V: Physical Register File Specifier Valid

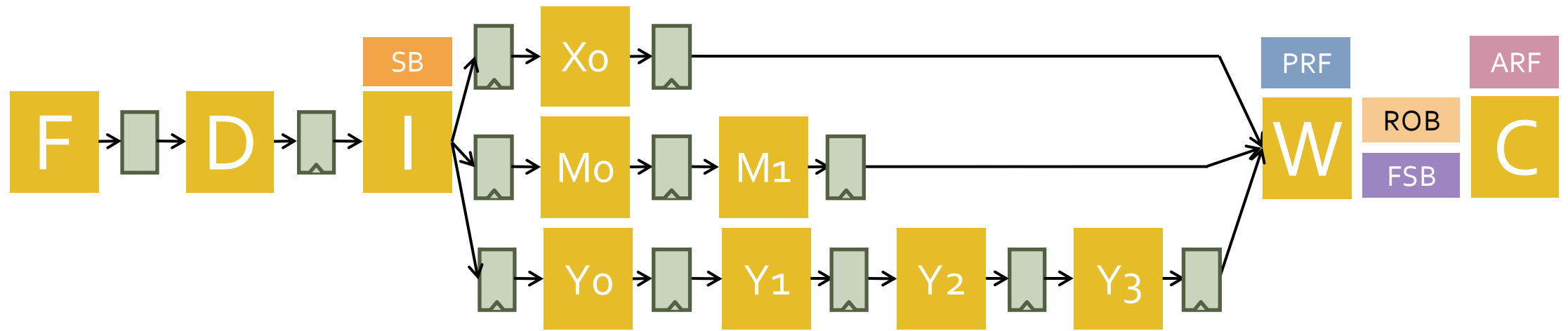
Preg: Physical Register File Specifier



Finished Store Buffer (FSB)

V	Op	Addr	Data
--			

- Only need one entry if we only support one memory instruction in flight at a time.
- Single Entry FSB makes allocation trivial.
- If support more than one memory instruction, we need to worry about Load/Store address aliasing.



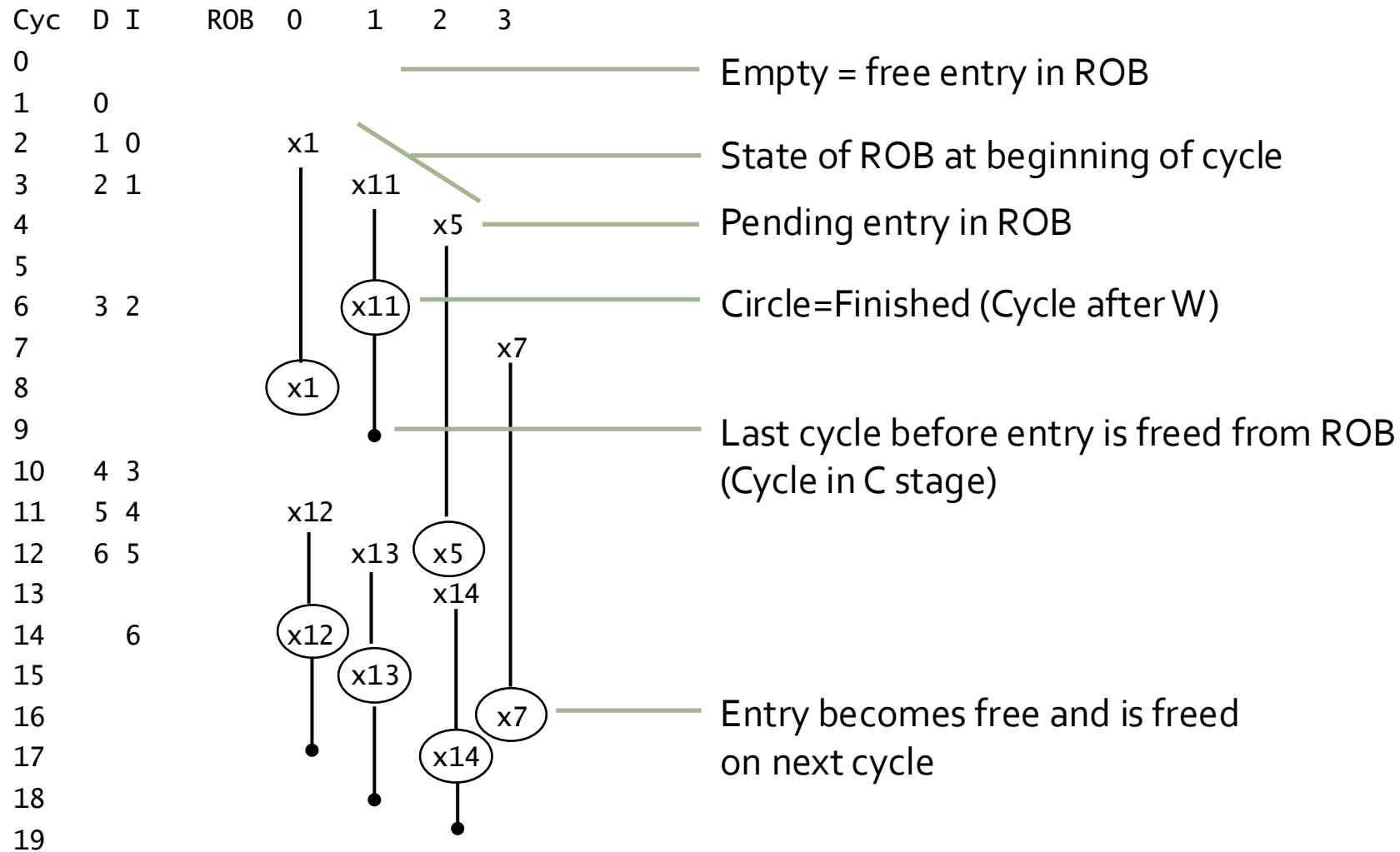
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

```

0 mul    x1, x2, x3
1 addi   x11,x10,1
2 mul    x5, x1, x4
3 mul    x7, x5, x6
4 addi   x12,x11,1
5 addi   x13,x12,1
6 addi   x14,x12,2

```

0	mul	x1, x2, x3	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	addi	x11,x10,1	F	D	I	Y0	Y1	Y2	Y3	W	C											
2	mul	x5, x1, x4		F	D	I	X0	W	r			C										
3	mul	x7, x5, x6			F	D	I	I	I	Y0	Y1	Y2	Y3	W	C							
4	addi	x12,x11,1				F	D	D	D	I	I	I	I	Y0	Y1	Y2	Y3	W	C			
5	addi	x13,x12,1					F	F	F	D	D	D	D	I	X0	W	r			C		
6	addi	x14,x12,2								F	F	F	F	D	I	X0	W	r		C		
														F	D	I	I	X0	W	r		C



What if First Instruction Causes an Exception?

0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	/
1	addi	x11,x10,1		F	D	I	X0	W	r	--	/
2	mul	x5, x1, x4			F	D	I	I	I	Y0	/
3	mul	x7, x5, x6				F	D	D	D	I	/
4	addi	x12,x11,1					F	F	F	D	/
										F	D I. . .

What About Branches?

Option 1

0 beq	x1, x0, target	F	D	I	X0	W	C	
1 addi	x11,x10,1		F	D	I	-		} Squash instructions earlier. Has more complexity. ROB needs many ports.
2 add	x5, x1, x4			F	D	-		
3 add	x7, x5, x6				F	-		
T addi	x12,x11,1					F	D	I . . .

Option 2

0 beq	x1, x0, target	F	D	I	X0	W	C	
1 addi	x11,x10,1		F	D	I	X0	/	} Squash instructions in ROB when Branch commits.
2 add	x5, x1, x4			F	D	I	/	
3 add	x7, x5, x6				F	D	/	
T addi	x12,x11,1					F	D	I . . .

Option 3

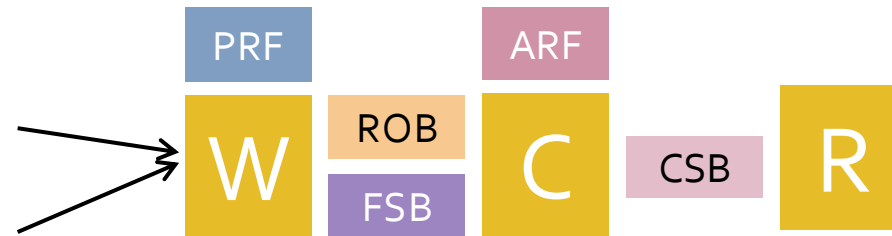
0 beq	x1, x0, target	F	D	I	X0	W	C	
1 addi	x11,x10,1		F	D	I	X0	W	/
2 add	x5, x1, x4			F	D	I	X0	W /
3 add	x7, x5, x6				F	D	I	X0 W /
T addi	x12,x11,1					F	D	I X0 W C

} Wait for speculative instructions to reach the Commit stage and squash in Commit stage

What About Branches?

- Three possible designs with decreasing complexity based on when to squash speculative instructions and de-allocate ROB entry:
 1. As soon as branch resolves
 2. When branch commits
 3. When speculative instructions reach commit
- Base design only allows one branch at a time. Second branch stalls in decode. Can add more bits to track multiple in-flight branches.

Avoiding Stalling Commit on Store Miss



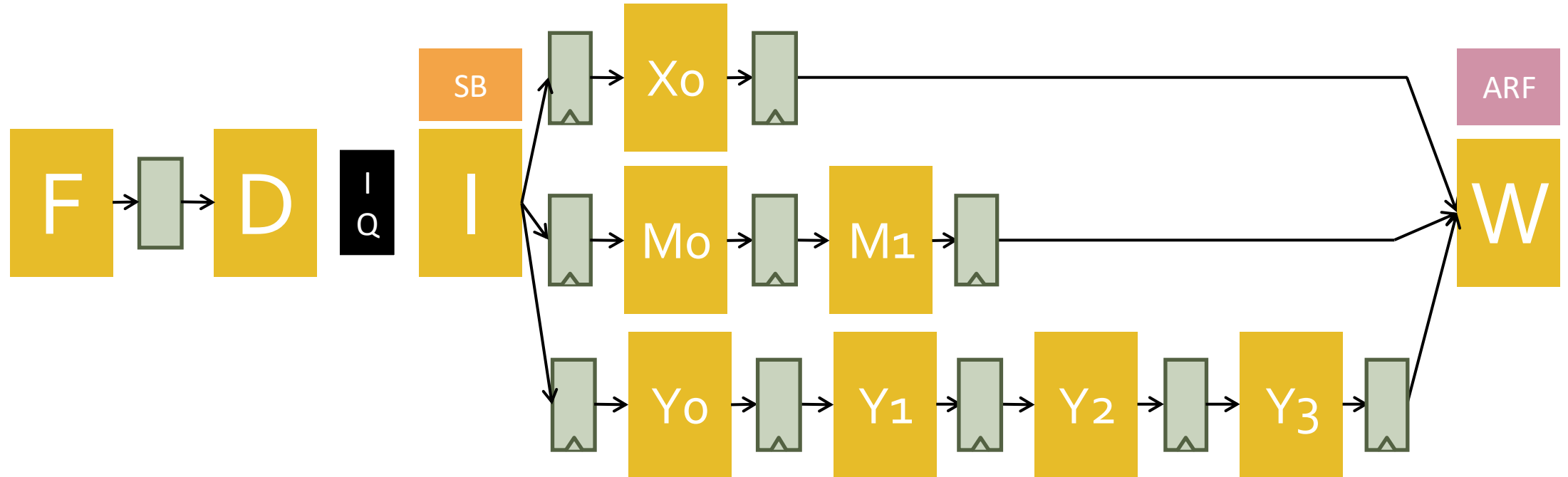
CSB=Committed Store Buffer

0 OpA	F	D	I	X	0	W	C				
1 SW		F	D	I	S	0	W	C	C	C	C
2 OpB			F	D	I	X	0	W	W	W	C
3 OpC				F	D	I	X	X	X	X	W
4 OpD					F	D	I	I	I	I	X

With Retire Stage

0 OpA	F	D	I	X	0	W	C				
1 SW		F	D	I	S	0	W	C	R	R	R
2 OpB			F	D	I	X	0	W	C		
3 OpC				F	D	I	X	W	C		
4 OpD					F	D	I	X	W	C	

IO3: In-order Frontend, Out-of-order Issue/Writeback/Commit



ARF

SB

IQ

W

R
R/W
R/W

W
W
W

Issue Queue (IQ)

Op	Imm	S	V	Dest	V	P	Src0	V	P	Src1

Op: Opcode

Imm.: Immediate

S: Speculative Bit

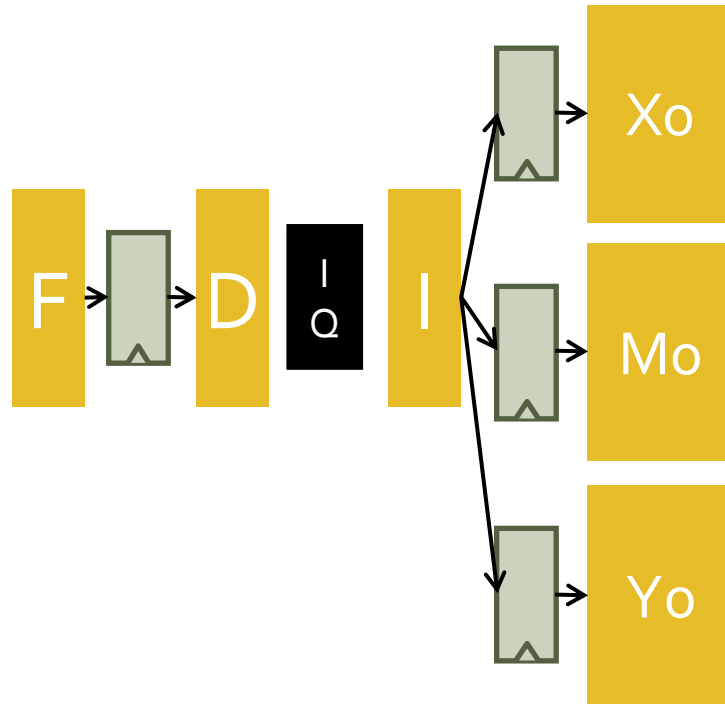
V: Valid (Instruction has corresponding Src/Dest)

P: Pending (Waiting on operands to be produced)

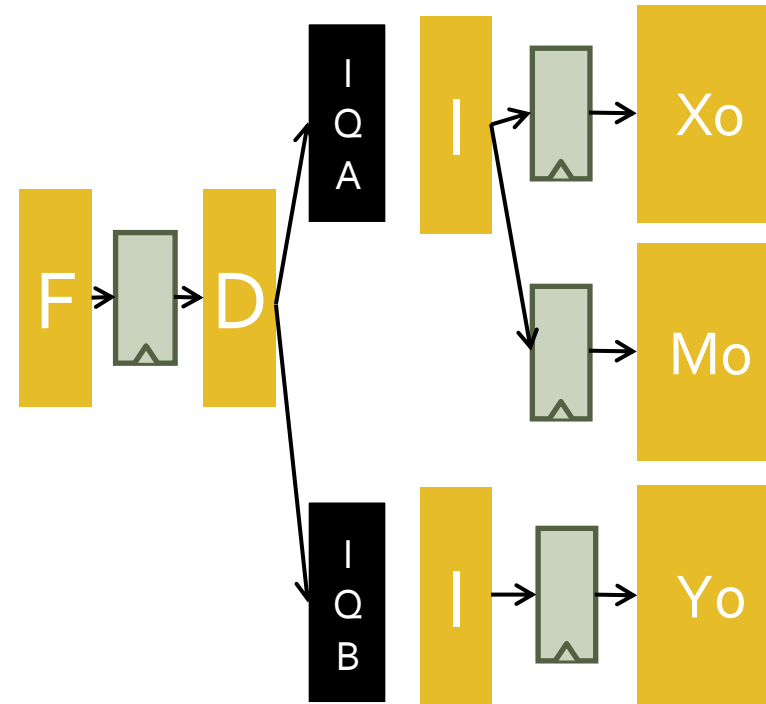
Instruction Ready = (!Vsrc0 || !Psrc0) && (!Vsrc1 || !Psrc1) && no structural hazards

- For high performance, factor in bypassing

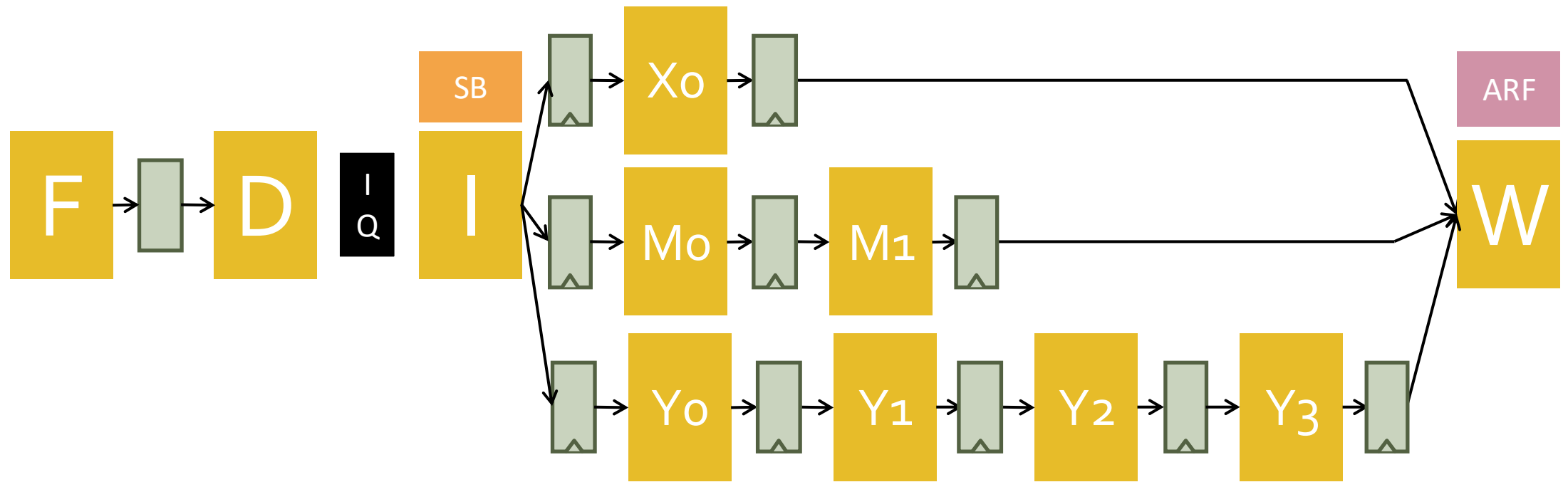
Centralized vs. Distributed Issue Queue



Centralized



Distributed



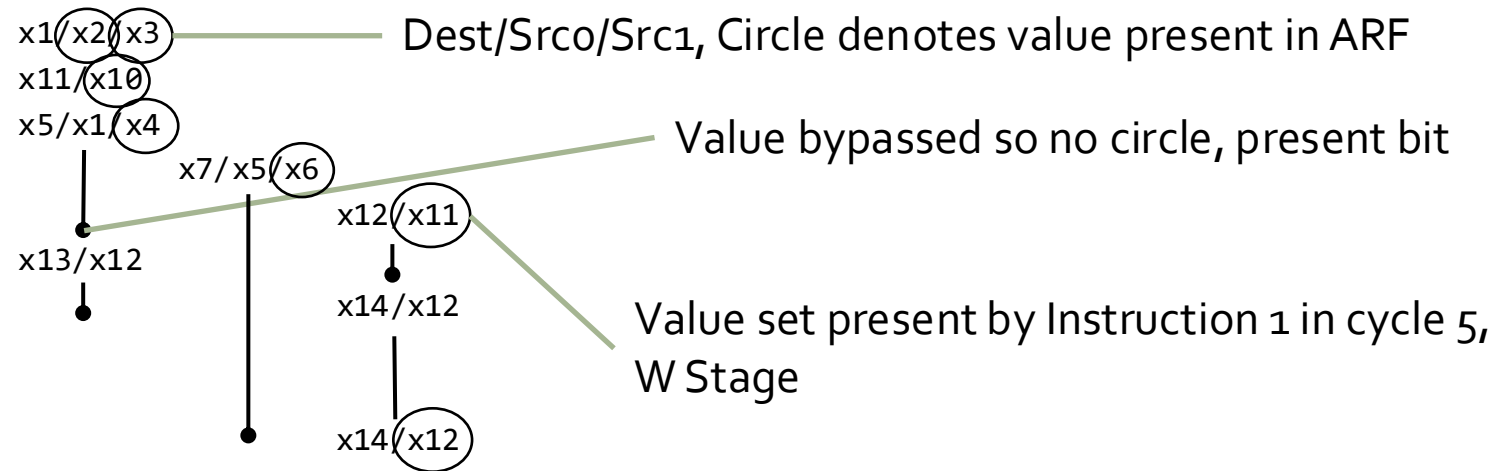
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

```

0 mul    x1, x2, x3
1 addi   x11,x10,1
2 mul    x5, x1, x4
3 mul    x7, x5, x6
4 addi   x12,x11,1
5 addi   x13,x12,1
6 addi   x14,x12,2
  
```

			0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W								
1	addi	x11,x10,1		F	D	I	X0	W										
2	mul	x5, x1, x4			F	D	i		I	Y0	Y1	Y2	Y3	W				
3	mul	x7, x5, x6				F	D	i					I	Y0	Y1	Y2	Y3	W
4	addi	x12,x11,1					F	D	i	I	X0	W						
5	addi	x13,x12,1						F	D	i	I	X0	W					
6	addi	x14,x12,2							F	D	i			I	X0	W		

Cyc	D	I	IQ	0	1	2
0						
1	0					
2	1	0				
3	2	1				
4	3					
5	4					
6	5	2				
7	6	4				
8		5				
9						
10		3				
11		6				
12						
13						
14						
15						

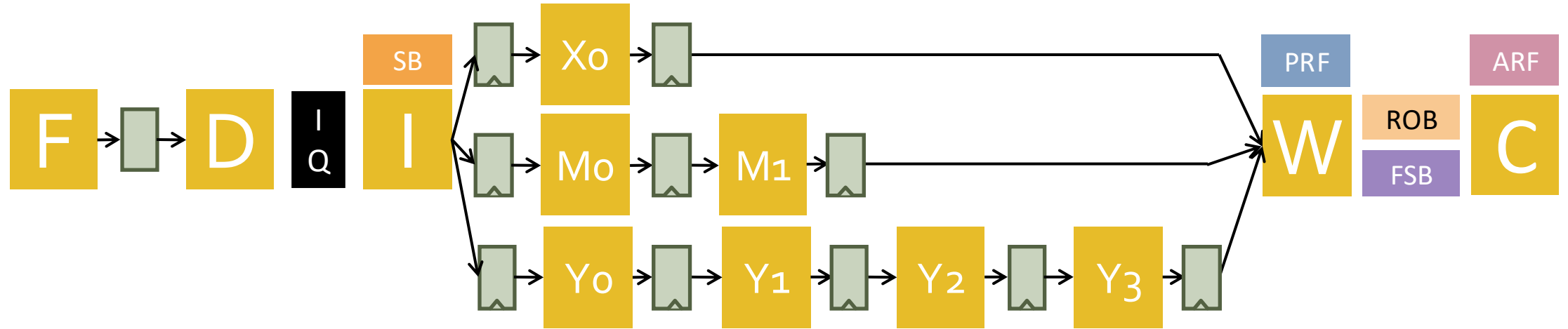


Assume All Instructions in Issue Queue

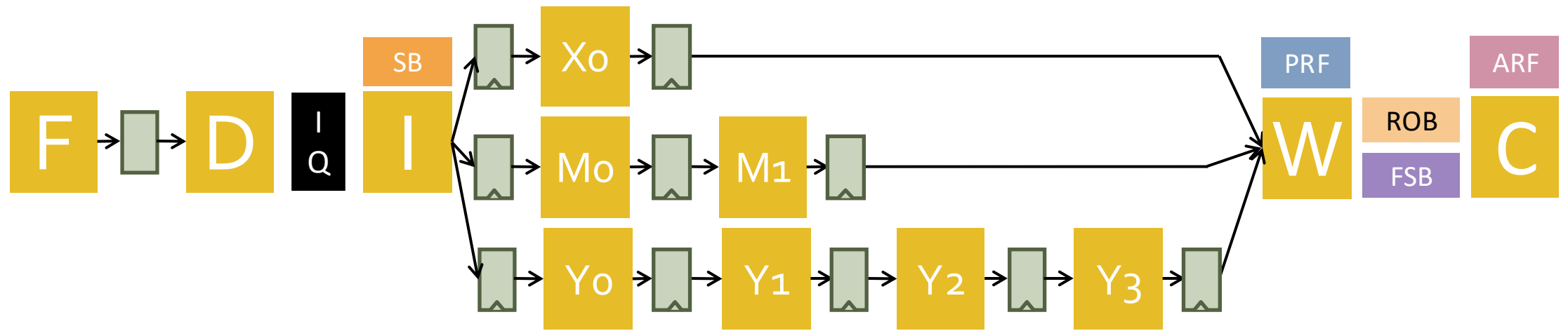
[illegible]

- Better performance than previous?

IO2I: In-order Frontend, Out-of-order Issue/Writeback, In-order Commit



	ARF	SB	PRF	ROB	FSB	IQ
ARF						
SB						
PRF						
ROB						
FSB						
IQ						



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul		x1,	x2,	x3															
1	addi		x11,	x10,	1															
2	mul		x5,	x1,	x4															
3	mul		x7,	x5,	x6															
4	addi		x12,	x11,	1															
5	addi		x13,	x12,	1															
6	addi		x14,	x12,	2															

			0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C											
1	addi	x11,x10,1		F	D	I	X0	W	r			C										
2	mul	x5, x1, x4			F	D	i		I	Y0	Y1	Y2	Y3	W	C							
3	mul	x7, x5, x6				F	D	i					I	Y0	Y1	Y2	Y3	W	C			
4	addi	x12,x11,1					F	D	i	I	X0	W	r							C		
5	addi	x13,x12,1						F	D	i	I	X0	W	r							C	
6	addi	x14,x12,2							F	D	i			I	X0	W	r					C

Difference?

0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C											
1	addi	x11,x10,1		F	D	I	X0	W	r			C										
2	mul	x5, x1, x4			F	D	i		I	Y0	Y1	Y2	Y3	W	C							
3	mul	x7, x5, x6				F	D	i					I	Y0	Y1	Y2	Y3	W	C			
4	addi	x12,x11,1					F	D	i	I	X0	W	r							C		
5	addi	x13,x12,1						F	D	i	I	X0	W	r							C	
6	addi	x14,x12,2							F	D	i		I	X0	W	r						C

Out-of-order 2-Wide Superscalar with 1 ALU

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C										
1	addi	x11,x10,1	F	D	I	X0	W	r			C										
2	mul	x5, x1, x4		F	D	i			I	Y0	Y1	Y2	Y3	W	C						
3	mul	x7, x5, x6		F	D	i						I	Y0	Y1	Y2	Y3	W	C			
4	addi	x12,x11,1			F	D	I	X0	W	r									C		
5	addi	x13,x12,1			F	D	i	I	X0	W	r									C	
6	addi	x14,x12,2				F	D	i	I	X0	W	r									C

Out-Of-Order (OOO)

Name	Frontend	Issue	Writeback	Commit	
I ₄	IO	IO	IO	IO	Fixed Length Pipelines Scoreboard
I ₂ O ₂	IO	IO	OOO	OOO	Scoreboard
I ₂ O ₁	IO	IO	OOO	IO	Scoreboard, Reorder Buffer, and Store Buffer
IO ₃	IO	OOO	OOO	OOO	Scoreboard and Issue Queue
IO ₂ I	IO	OOO	OOO	IO	Scoreboard, Issue Queue, Reorder Buffer, and Store Buffer

Superscalar Intel Processors

Computer Organization and Design RISC-V Edition 2nd Edition Figure 4.74

Microprocessor	Year	Clock Rate	Pipeline Stages	Issue Width	Out-of-Order / Speculation	Cores/Chip	Power
Intel 486	1989	25 MHz	5	1	No	1	5W
Intel Pentium	1993	66 MHz	5	2	No	1	10W
Intel Pentium Pro	1997	200 MHz	10	3	Yes	1	29W
Intel Pentium 4 Willamette	2001	2000 MHz	22	3	Yes	1	75W
Intel Pentium 4 Prescott	2004	3600 MHz	31	3	Yes	1	103W
Intel Core	2006	3000 MHz	14	4	Yes	2	75W
Intel Core i7 Nehalem	2008	3600 MHz	14	4	Yes	2-4	87W
Intel Core Westmere	2010	3730 MHz	14	4	Yes	6	130W
Intel Core i7 Ivy Bridge	2012	3400 MHz	14	4	Yes	6	130W
Intel Core Broadwell	2014	3700 MHz	14	4	Yes	10	140W
Intel Core i9 Skylake	2016	3100 MHz	14	4	Yes	14	165W
Intel Ice Lake	2018	4200 MHz	14	4	Yes	16	185W

- Pentium 4: Marketing demanded higher clock rate $\Rightarrow$ deeper pipelines & higher power consumption
- Afterwards: Multi-core processors

ARM Cortex-A53 vs. Intel Core i7

Characteristic	ARM A53	Intel Core i7 920
Market	Personal Mobile Device	Server, Cloud
Thermal Design Power	100 milliWatts (1 core @ 1 GHz)	130 Watts
Clock Rate	1.5 GHz	2.66 GHz
Cores/Chip	4 (configurable)	4
Floating Point?	Yes	Yes
Multiple Issue?	Dynamic	Dynamic
Peak Instructions/Clock Cycle	2	4
Pipeline Stages	8	14
Pipeline Schedule	Static In-order	Dynamic Out-of-order with Speculation
Branch Prediction	Hybrid	2-level
1st Level Caches/Core	16-64 KiB I, 16-64 KiB D	32 KiB I, 32 KiB D
2nd Level Cache/Core	128-2048 KiB (shared)	256 KiB (per core)
3rd Level Cache (Shared)	(platform dependent)	2-8 MiB

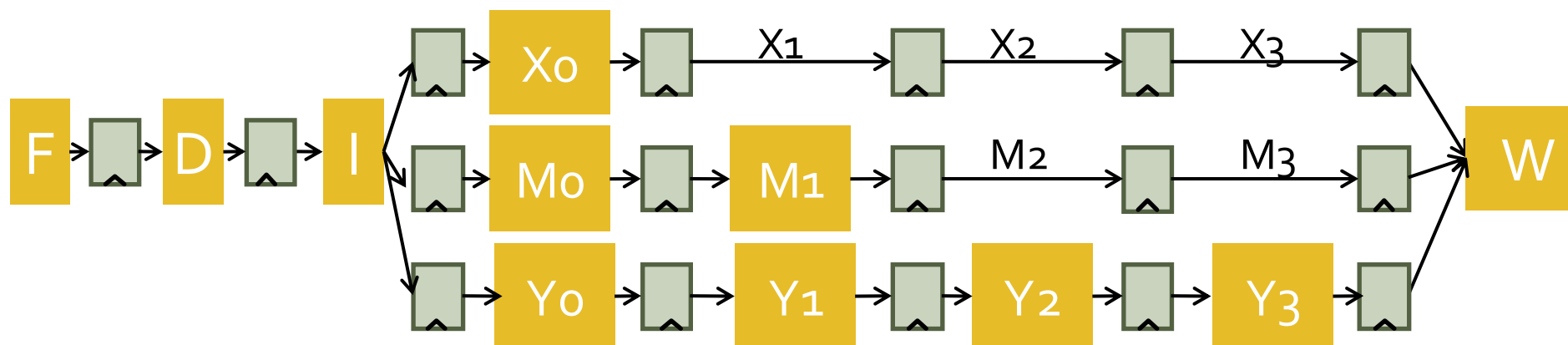
Agenda

- Out-of-Order Processors
- Speculation and Branches
- Register Renaming
- Memory Disambiguation

Speculation and Branches: I4

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W											
1	addi	x4, x5, 1		F	D	I	X0	X1	X2	X3	W										
2	mul	x6, x1, x4			F	D	I	I	I	Y0	Y1	Y2	Y3	W							
3	beq	x6, x0, Target				F	D	D	D	I	I	I	I	X0	X1	X2	X3	W			
4	addi	x8, x9, 1					F	F	F	D	D	D	D	I	--	--	--	--	--		
5	addi	x10, x11, 1							F	F	F	F	D		--	--	--	--	--	--	
6	addi	x12, x13, 1											F		--	--	--	--	--	--	--
T														F	D	I	.	.	.		

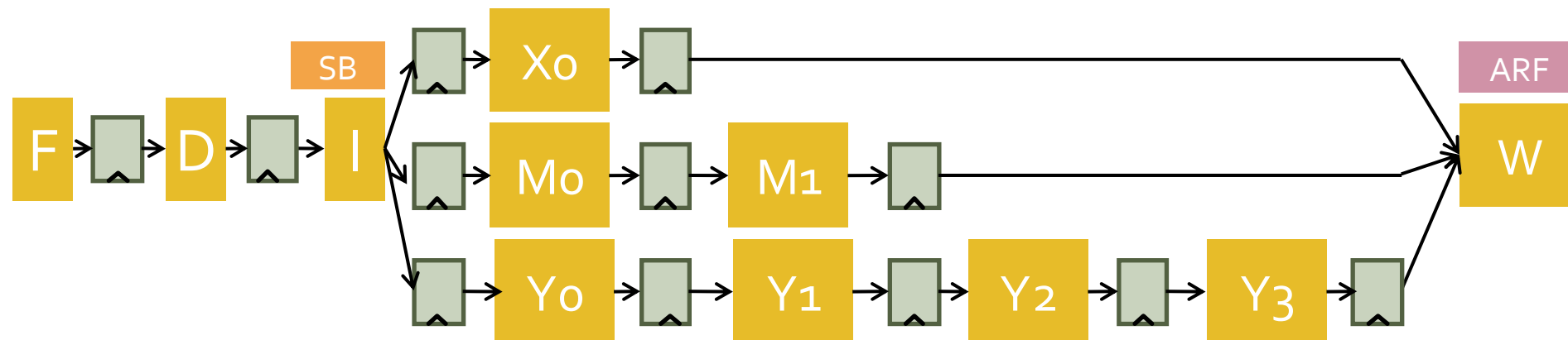
- No speculative instructions commit state



Speculation and Branches: I2O2

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W											
1	addi	x4, x5, 1		F	D	I	X0	W													
2	mul	x6, x1, x4			F	D	I	I	I	Y0	Y1	Y2	Y3	W							
3	beq	x6, x0, Target				F	D	D	D	I	I	I	I	X0	W						
4	addi	x8, x9, 1					F	F	F	D	D	D	D	I	--	--					
5	addi	x10, x11, 1							F	F	F	F	D	--	--	--					
6	addi	x12, x13, 1											F	--	--	--	--				
T														F	D	I	.	.	.		

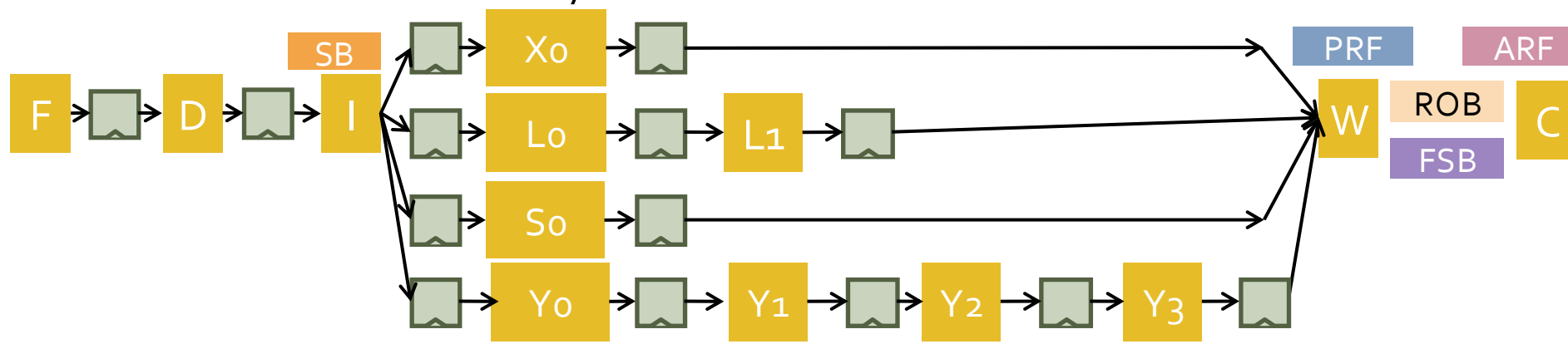
- No speculative instructions commit state



Speculation and Branches: I2OI

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C										
1	addi	x4, x5, 1		F	D	I	X0	W	r				C								
2	mul	x6, x1, x4			F	D	I	I	I	Y0	Y1	Y2	Y3	W	C						
3	beq	x6, x0, Target				F	D	D	D	I	I	I	I	X0	W	C					
4	addi	x8, x9, 1					F	F	F	D	D	D	D	I	--	--	--				
5	addi	x10, x11, 1							F	F	F	F	D	--	--	--	--				
6	addi	x12, x13, 1											F	--	--	--	--	--			
T														F	D	I	.	.	.		

- Must squash instructions in pipeline after branch to prevent PRF write
- Can remove from ROB immediately or wait until commit

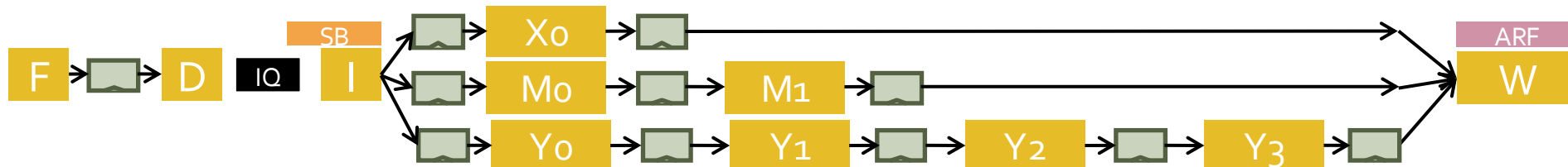


Speculation and Branches: IO₃

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W											
1	addi	x4, x5, 1		F	D	I	X0	W													
2	mul	x6, x1, x4			F	D	i		I	Y0	Y1	Y2	Y3	W							
3	beq	x6, x0, Target				F	D	i					I	X0	W						
4	addi	x8, x9, 1					F	D	i	I	X0	W									
5	addi	x10, x11, 1						F	D	i	I	X0	W								
6	addi	x12, x13, 1							F	D	i			I	X0	W					
7	???								F	D											
8	???									F	D										
9	???										F	D									
10	???											F	D								
11	???												F	D							
T														F	D	I	.	.	.		

Speculative
Instructions
Wrote to ARF

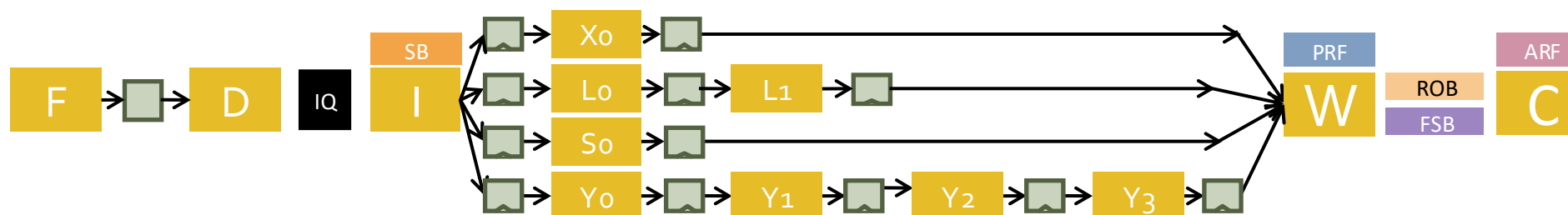
- No control speculation for IO₃
- Could stall on branch



Speculation and Branches: IO2I

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C										
1	addi	x4, x5, 1		F	D	I	X0	W	r			C									
2	mul	x6, x1, x4			F	D	i		I	Y0	Y1	Y2	Y3	W	C						
3	beq	x6, x0, Target				F	D	i					I	X0	W	C					
4	addi	x8, x9, 1					F	D	i	I	X0	W	r	--							
5	addi	x10, x11, 1						F	D	i	I	X0	W	--							
6	addi	x12, x13, 1							F	D	i			--							
7	???								F	D				--							
8	???									F	D			--							
9	???										F	D		--							
10	???											F	D	--							
11	???												F	--							
T														--	D						

Need to clean up
Speculative state
In PRF. Needs
selective rollback



Speculation and Branches: IO2I

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C										
1	addi	x4, x5, 1		F	D	I	X0	W	r			C									
2	mul	x6, x1, x4			F	D	i		I	Y0	Y1	Y2	Y3	W	C						
3	beq	x6, x0, Target				F	D	i					I	X0	W	C					
4	addi	x8, x9 ,1					F	D	i	I	X0	W	r								
5	addi	x10,x11,1						F	D	i	I	X0	W	r							
6	addi	x12,x13,1							F	D	i			I	X0						
7	???									F	D										
8	???										F	D									
9	???											F	D								
10	???												F	D							
11	???													F	D						
12	???														F						
13	???																				
T																					

- Copy ARF to PRF on Mispredict

Speculative
Instructions
Wrote to PRF
Not ARF

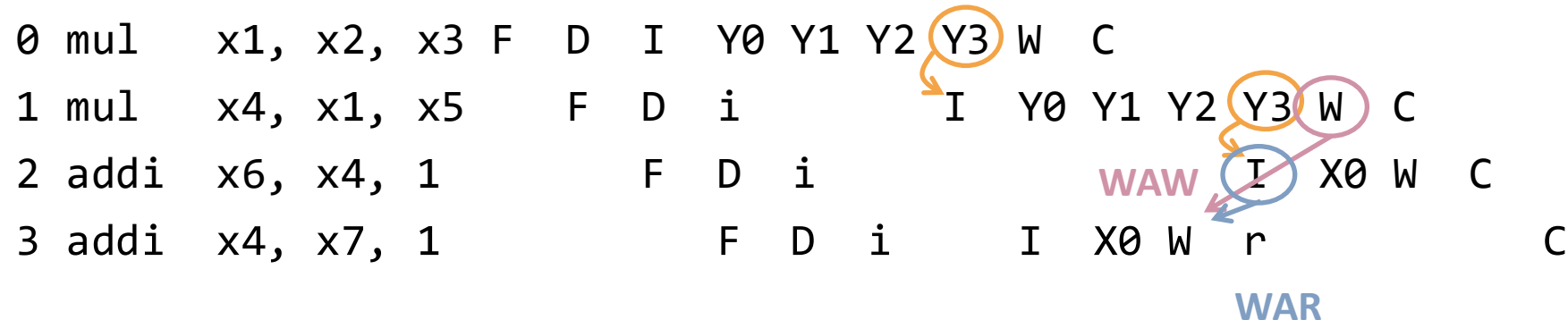
Agenda

- Out-of-Order Processors
- Speculation and Branches
- **Register Renaming**
- Memory Disambiguation

WAW and WAR “Name” Dependencies

- WAW and WAR are not “True” data dependencies
- RAW is “True” data dependency because reader needs result of writer
- “Name” dependencies exist because we have limited number of “Names” (register specifiers or memory addresses)

Breaking all “Name” Dependencies (Causes problems)



Adding More Registers

Breaking all “Name” Dependencies

0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C						
1	mul	x4, x1, x5		F	D	i			I	Y0	Y1	Y2	Y3	W	C		
2	addi	x6, x4, 1			F	D	i						I	X0	W	C	
3	addi	x4, x7, 1				F	D	i		I	X0	W	r				C

IO2I Microarchitecture Conservatively Stalls

PC	Op	Op1	Op2	Op3	Op4	Op5	Op6	Op7	Op8	Op9	Op10	Op11	Op12	Op13	Op14	Op15	Op16	Op17	Op18	Op19	Op20	Op21	Op22	Op23	Op24	Op25	Op26	Op27	Op28	Op29	Op30	Op31		
0	mul	x1,	x2,	x3	F	D	I	Y0	Y1	Y2	Y3	W	C																					
1	mul	x4,	x1,	x5		F	D	i			I	Y0	Y1	Y2	Y3	W	C																	
2	addi	x6,	x4,	1			F	D	i							I	X0	W	C															
3	addi	x4,	x7,	1				F	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	I	X0	W	C				

Manual Register Renaming. What if we could use more registers? Second X₄ Write to X₈?

0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C						
1	mul	x4, x1, x5		F	D	i				I	Y0	Y1	Y2	Y3	W	C	
2	addi	x6, x4, 1			F	D	i							I	X0	W	C
3	addi	x8, x7, 1				F	D	i			I	X0	W	r			C

How many Instructions can be in the pipeline?

- Throughput is limited by number of instructions in flight, but which feature of an ISA limits the number of instructions in the pipeline?

Register Renaming

- Adding more “Names” (registers/memory) removes dependence, but architecture namespace is limited.
 - Registers: Larger namespace requires more bits in instruction encoding. 32 registers = 5 bits, 128 registers = 7 bits.
- **Register Renaming:** Change naming of registers in hardware to eliminate WAW and WAR hazards

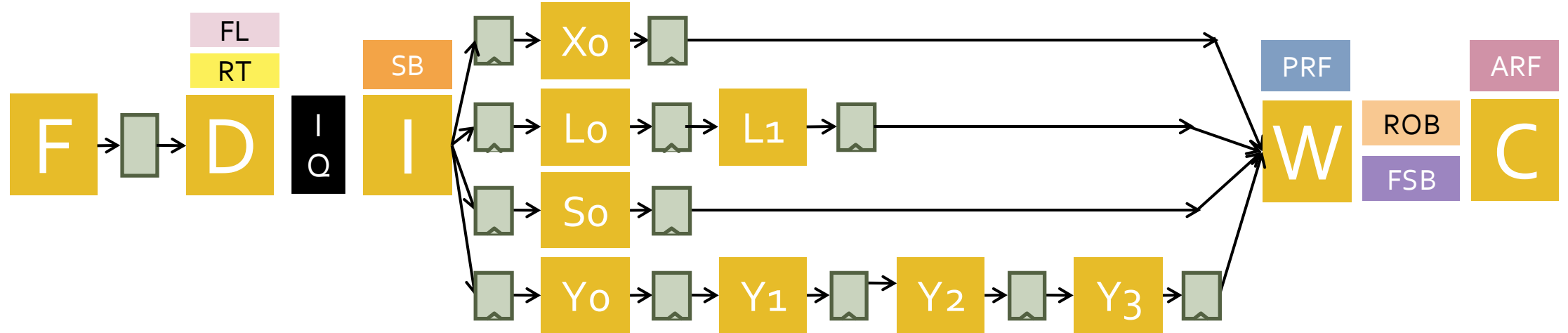
Floating Point pipelines often cannot be kept filled with small number of registers.

→ IBM 360 had only 4 Floating Point Registers

Register Renaming Overview

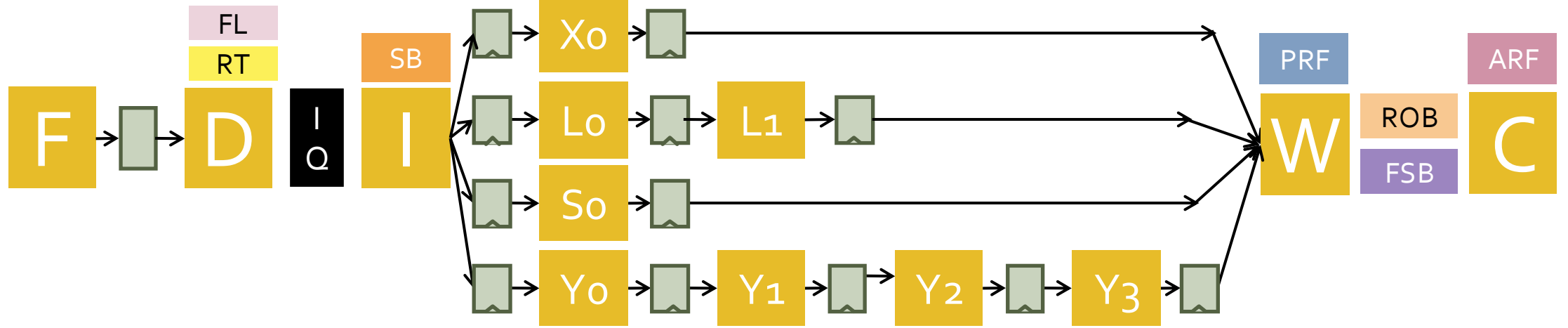
- 2 schemes
 - Pointers in the Issue Queue/ReOrder Buffer
 - Values in the Issue Queue/ReOrder Buffer
- IO2I uses pointers in IQ and ROB therefore start with that design

IO2I: Register Renaming with Pointers in IQ and ROB



- All data structures same as in IO2I Except:
 - Add two fields to ROB
 - Add Rename Table (RT) and Free List (FL) of registers
- Increase size of PRF to provide more register “Names”

IO2: Register Renaming with Pointers in IQ and ROB



Cache	Read	Write	Read/Write	Write-Back	Write-Through
ARF					W
SB			R/W		W
PRF			R		W
ROB	R/W				W
FSB					W
IQ	W		R/W		W
FL	R/W				W
RT	R/W				W

48

Modified Reorder Buffer (ROB)

State	S	ST	V	Preg	Areg	Ppreg
--						
P						
F						
P						
P						
F						
P						
P						
--						

State: {Empty (--), Pending, Finished}

S: Speculative

ST: Store bit

V: Destination is valid

Preg: Physical Register File Specifier

Areg: Architectural Register File Specifier

Ppreg: Previous Physical Register

Rename Table (RT)

	P	Preg
x1		
x2		
x3		
...		
x31		

P: Pending, Write to Destination in flight

Preg: Physical Register Architectural Register maps to

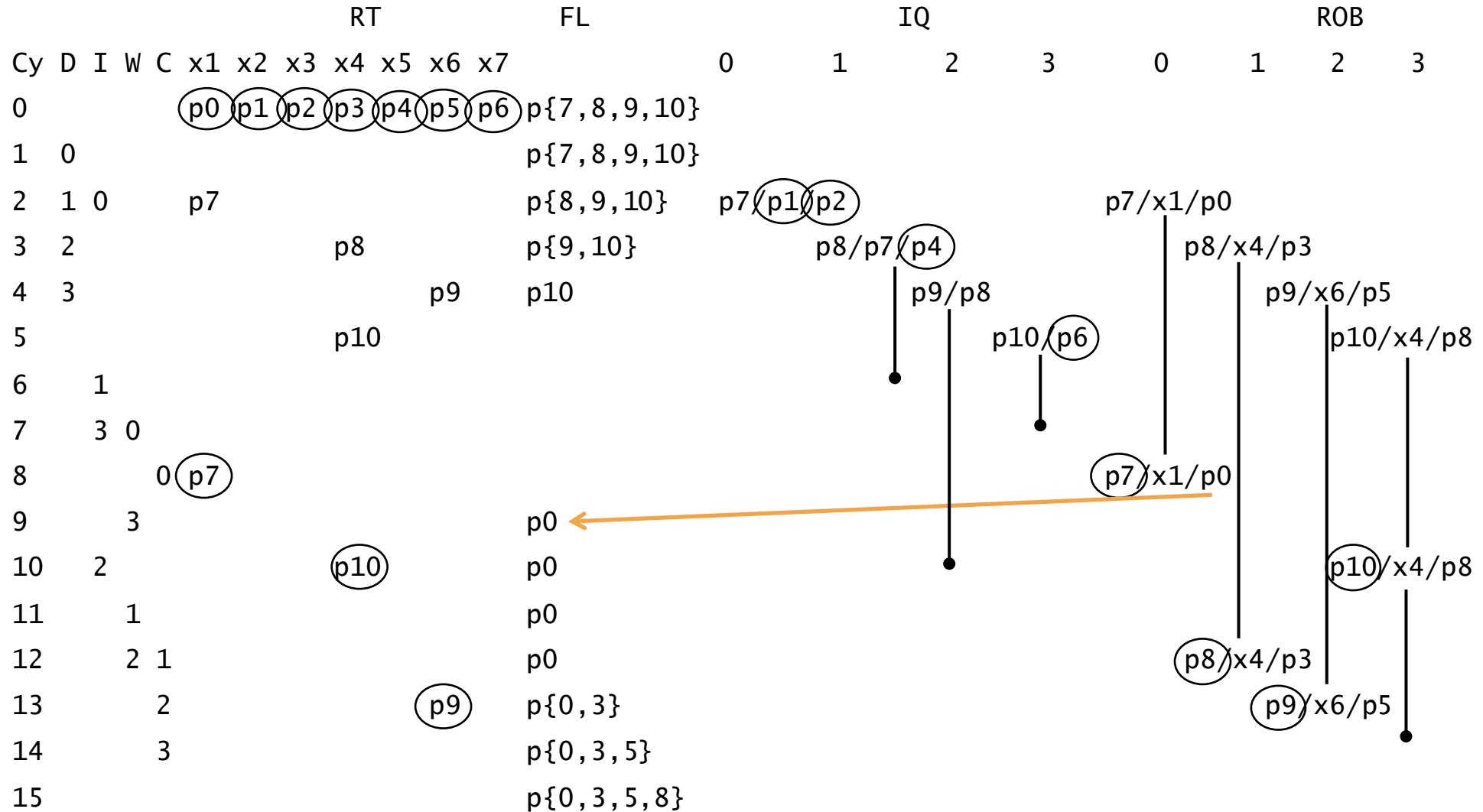
Free List (FL)

	Free
p1	
p2	
p3	
...	
pN	

Free: Register is free for renaming

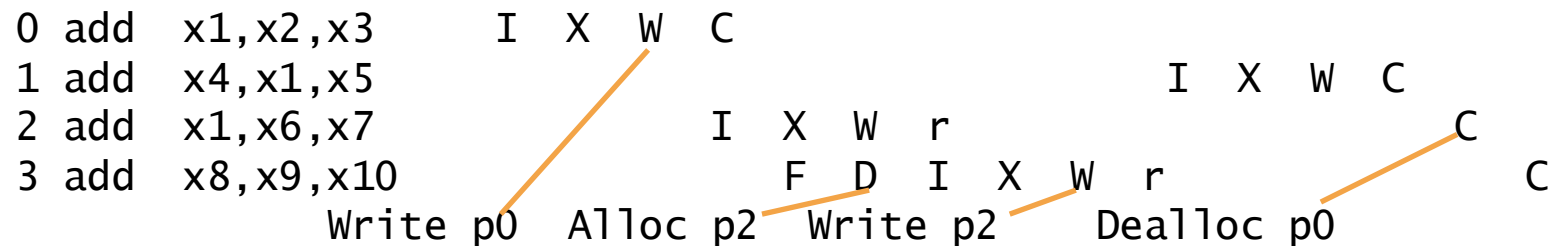
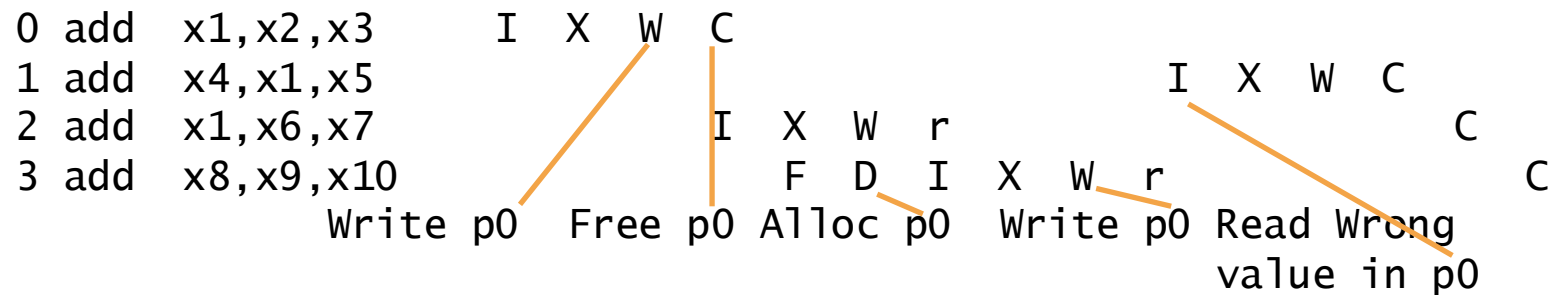
If Free == 0, physical register is in use and cannot be used for renaming

				0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C								
1	mul	x4, x1, x5		F	D	i				I	Y0	Y1	Y2	Y3	W	C			
2	addi	x6, x4, 1			F	D	i							I	X0	W	C		
3	addi	x4, x7, 1				F	D	i			I	X0	W	r				C	



Freeing Physical Registers

add x1,x2,x3 $\Leftarrow$ Assume Arch. Reg X1 maps to Phys. Reg p0
 add x4,x1,x5
 add x1,x6,x7 $\Leftarrow$ Next write of Arch Reg X1, Mapped to Phys. Reg p1
 add x8,x9,x10

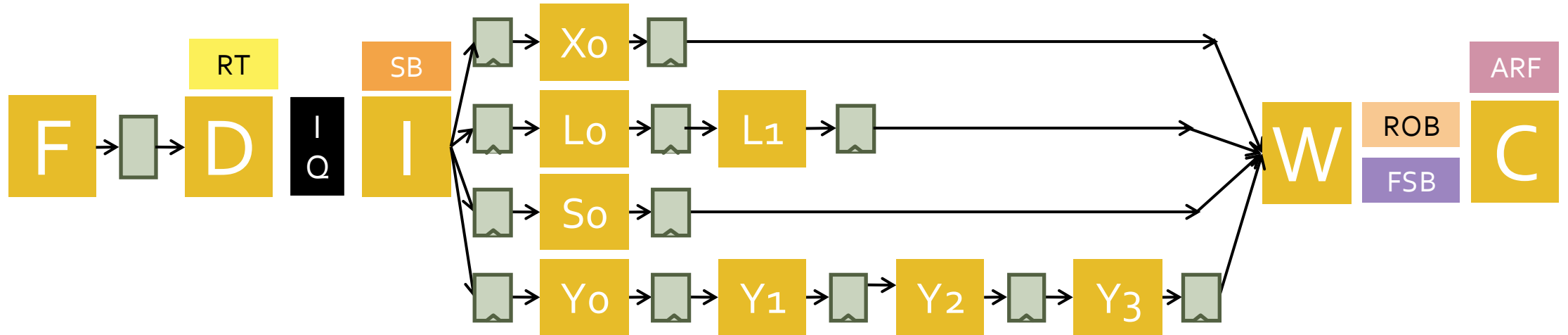


- If Arch. Reg Xi mapped to Phys. Reg pj, we can free pj when the next instruction that writes Xi commits

Unified Physical/Architectural Register File

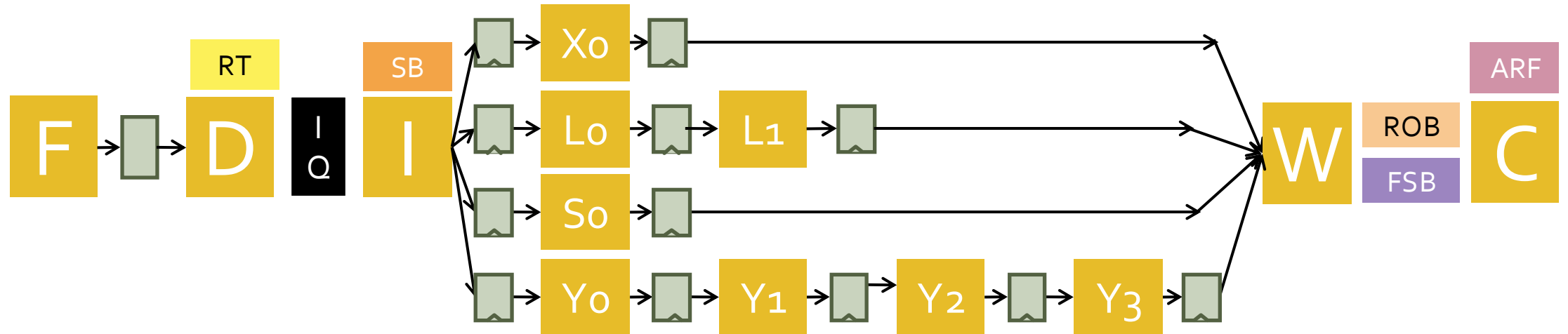
- Combine PRF and ARF into one register file
- Replace ARF with Architectural Rename Table
- Instead of copying **Values**, Commit stage copies Preg pointer into appropriate entry of Architectural Rename Table
- Unified Physical/Architectural Register file can be smaller than separate

IO2: Register Renaming with Values in IQ and ROB



- All data structures same as previous Except:
 - Modified ROB (Values instead of Register Specifier)
 - Modified RT
 - Modified IQ
 - No FL
 - No PRF, values merged into ROB

IO2: Register Renaming with Values in IQ and ROB



ARF	R					W
SB		R/W				W
ROB	R/W					R/W
FSB						R/W
IQ	W	R/W				W
RT	R/W					W

Modified Reorder Buffer (ROB)

State	S	ST	V	Value	Areg
--					
P					
F					
P					
P					
F					
P					
P					
--					

State: {Empty (--), Pending, Finished} **V:** Destination is valid

S: Speculative

Value: Actual Register Value

ST: Store bit

Areg: Architectural Register File Specifier

Modified Issue Queue (IQ)

Op	Imm	S	V	Dest	V	P	Src0	V	P	Src1

Op: Opcode

Imm.: Immediate

S: Speculative Bit

V: Valid (Instruction has corresponding Src/Dest)

P: Pending (Waiting on operands to be produced)

If Pending, Source Field contains index into ROB. Like a Preg identifier

On Commit, Source Field contains value

Modified Rename Table (RT)

	V	P	Preg
x1			
x2			
x3			
...			
x31			

V: Valid Bit

P: Pending, Write to Destination in flight

Preg: Index into ROB

V:

If V == 0:

Value in ARF is up to date

If V == 1:

Value is in-flight or in ROB

P:

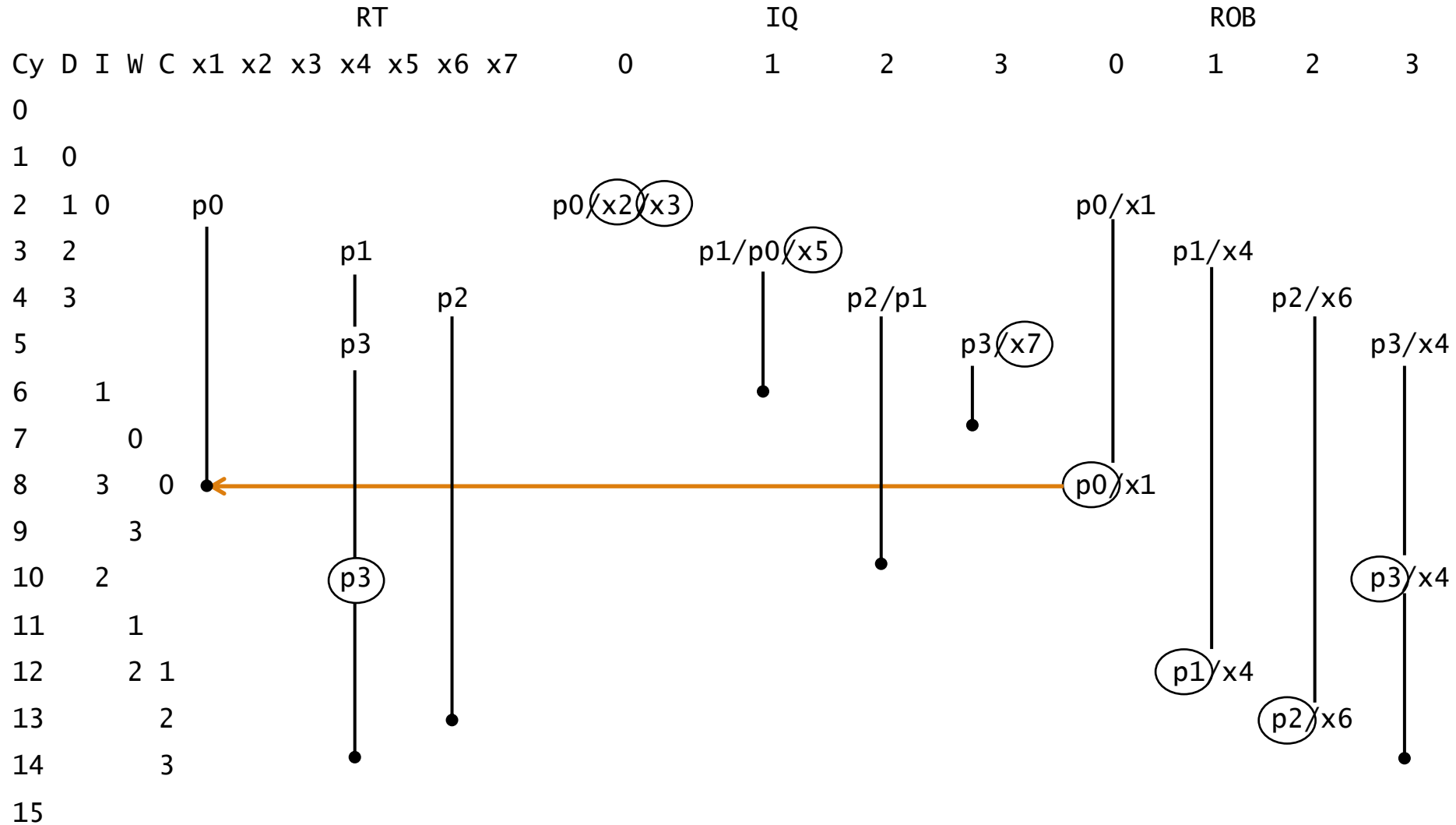
If P == 0:

Value is in ROB

if P == 1:

Value is in flight

						0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	mul	x1, x2, x3	F	D	I	Y0	Y1	Y2	Y3	W	C										
1	mul	x4, x1, x5		F	D	i				I	Y0	Y1	Y2	Y3	W	C					
2	addi	x6, x4, 1			F	D	i								I	X0	W	C			
3	addi	x4, x7, 1				F	D	i		I	X0	W	r						C		



Agenda

- Out-of-Order Processors
- Speculation and Branches
- Register Renaming
- Memory Disambiguation

Memory Disambiguation

```
sw x1, 0(x2)
```

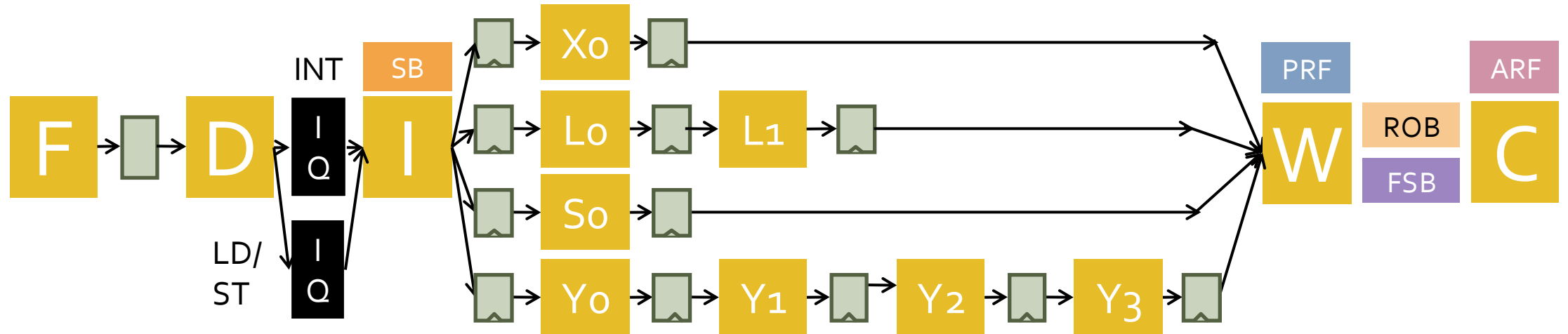
```
lw x3, 0(x4)
```

When can we execute the load?

In-Order Memory Queue

- Execute all loads and stores in program order
 - => Load and store cannot leave IQ for execution until all previous loads and stores have completed execution
- Can still execute loads and stores speculatively, and out-of-order with respect to other (non-memory) instructions
- Need a structure to handle memory ordering...

IO2I: With In-Order LD/ST IQ



Conservative OOO Load Execution

sw x1, 0(x2)

lw x3, 0(x4)

- Split execution of store instruction into two phases: address calculation and data write
- Can execute load before store, if addresses known and $x_4 \neq x_2$
- Each load address compared with addresses of all previous uncommitted stores (can use partial conservative check i.e., bottom 12 bits of address)
- Don't execute load if any previous store address not known

(MIPS R10K, 16 entry address queue)

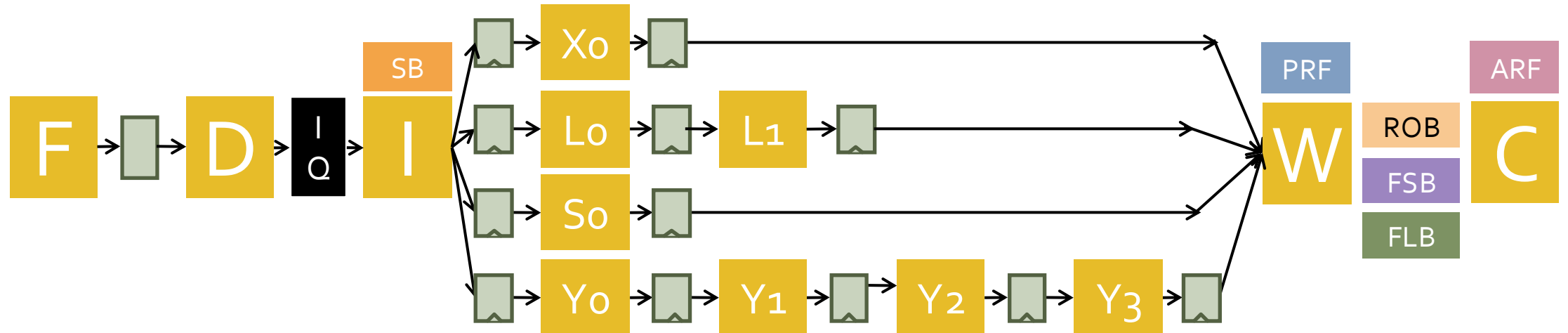
Address Speculation

```
sw x1, 0(x2)
```

```
lw x3, 0(x4)
```

- Guess that $x_4 \neq x_2$
- Execute load before store address known
- Need to hold all completed but uncommitted load/store addresses in program order
- If subsequently find $x_4 = x_2$, squash load and *all* dependent instructions
=> Large penalty for inaccurate address speculation

IO2I: With OOO Load and Stores



Memory Dependence Prediction

(Alpha 21264)

```
sw x1, 0(x2)
lw x3, 0(x4)
```

- Guess that $x_4 \neq x_2$ and execute load before store
- If later find $x_4 = x_2$, squash load and all following instructions, but mark load instruction as *store-wait*
- Subsequent executions of the same load instruction will wait for all previous stores to complete
- Periodically clear **store-wait** bits

Recap

- Out-of-Order Processors
- Speculation and Branches
- Register Renaming
- Memory Disambiguation

Acknowledgements

- These slides contain material developed and copyright by:
 - Arvind (MIT)
 - Krste Asanovic (MIT/UCB)
 - Joel Emer (Intel/MIT)
 - James Hoe (CMU)
 - John Kubiatowicz (UCB)
 - David Patterson (UCB)
 - Christopher Batten (Cornell)
 - David Wentzlaff (Princeton)
- MIT material derived from course 6.823
- UCB material derived from course CS252
- Cornell material derived from course ECE 4750
- Princeton material derived from course ECE 475